SIMATS ENGINEERING

HARIKA(192572140)

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

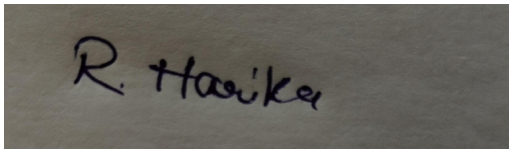
Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**

7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	**5**
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I R.HARIKA(192572140)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1.Given Relations

1. Apply Relational Algebra Operations (σ, π, ∪, −, ×, ⋈)
Relations

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	50000
102	Priya	D2	60000
103	Arjun	D1	55000
104	Sneha	D3	45000

DEPARTMENT

DeptID	DeptName
D1	HR
D2	IT
D3	Finance

Relational Algebra Expressions

1. Selection (σ) – Employees with salary greater than 50,000
σ Salary > 50000 (EMPLOYEE)

Output

EmpID	EmpName	DeptID	Salary
102	Priya	D2	60000
103	Arjun	D1	55000

2. Projection (π) – Display only Employee Name and Salary
π EmpName, Salary (EMPLOYEE)

Output

EmpName	Salary
Rahul	50000
Priya	60000
Arjun	55000
Sneha	45000

3. Union (∪)
Relation A

DeptID
D1
D2

Relation B

DeptID
D2
D3

A ∪ B

Output

DeptID
D1
D2
D3

4. Set Difference (−)
A − B
Output

DeptID
D1

5. Cartesian Product (×)
EMPLOYEE × DEPARTMENT

Result: Every employee is combined with every department (4 × 3 = 12 rows).

6. Join (⋈)

EMPLOYEE ⋈ EMPLOYEE.DeptID = DEPARTMENT.DeptID DEPARTMENT

Output

EmpID	EmpName	DeptName	Salary
101	Rahul	HR	50000
102	Priya	IT	60000
103	Arjun	HR	55000
104	Sneha	Finance	45000

2. SQL DDL Statements to Create STUDENT and DEPARTMENT Tables with Appropriate Constraints

Create the DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

Explanation

DeptID INT PRIMARY KEY

Uniquely identifies each department.
Cannot contain NULL values.

DeptName VARCHAR(50)

Stores the department name.
NOT NULL ensures every department has a name.
UNIQUE prevents duplicate department names.

CREATE THE STUDENT TABLE

```
CREATE TABLE STUDENT (
   SID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   DOB DATE NOT NULL,
   DeptID INT,
   CONSTRAINT FK_Dept
      FOREIGN KEY (DeptID)
      REFERENCES DEPARTMENT(DeptID)
      ON DELETE CASCADE
      ON UPDATE CASCADE
```

COMPLETE SQL PROGRAM

```
CREATE TABLE DEPARTMENT (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE STUDENT (
   SID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   DOB DATE NOT NULL,
   DeptID INT,
   CONSTRAINT FK_Dept
      FOREIGN KEY (DeptID)
      REFERENCES DEPARTMENT(DeptID)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

INSERT RECORDS

```
INSERT INTO DEPARTMENT VALUES
```

(101, 'Computer Science'),
(102, 'Information Technology'),
(103, 'Electronics');

INSERT INTO STUDENT VALUES
(1, 'Rahul', '2004-05-12', 101),
(2, 'Priya', '2003-08-25', 102),
(3, 'Arjun', '2004-02-15', 101),
(4, 'Sneha', '2003-11-10', 103);

Output of DEPARTMENT Table

DeptID	DeptName
101	Computer Science
102	Information Technology
103	Electronics

Output of STUDENT Table

SID	Name	DOB	DeptID
1	Rahul	2004-05-12	101
2	Priya	2003-08-25	102
3	Arjun	2004-02-15	101
4	Sneha	2003-11-10	103

3. SQL Queries Using GROUP BY, HAVING and Aggregate Functions

SALES Table

SaleID	Product		Amount
1	Laptop		50000
2		Laptop	45000
3		Mobile	25000
4		Mobile	30000
5		Tablet	20000

COUNT

SELECT Product, COUNT(*) AS TotalSales
FROM SALES
GROUP BY Product;

(a) COUNT
SELECT Product, COUNT(*) AS TotalSales
FROM SALES
GROUP BY Product;

	TotalSales
Laptop	2
Mobile	2

SaleID	Product	Amount
Tablet	1	

b)SUM

```
SELECT Product, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Product;
```

(c) AVG

```
SELECT Product, AVG(Amount) AS AverageAmount
FROM SALES
GROUP BY Product;
```

(d) MAX

```
SELECT Product, MAX(Amount) AS HighestSale
FROM SALES
GROUP BY Product;
```

(e) MIN

```
SELECT Product, MIN(Amount) AS LowestSale
FROM SALES
GROUP BY Product;
```

(f) HAVING Clause

Products whose total sales exceed ₹50,000.

```
SELECT Product, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Product
HAVING SUM(Amount) > 50000;
```

Output

Product	TotalAmount
Laptop	95000
Mobile	55000

4. CORRELATED SUB-QUERY

EMPLOYEE Table

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	50000
102	Priya	D2	60000
103	Arjun	D1	55000
104	Sneha	D2	45000

SQL Query

```
SELECT EmpID, EmpName, DeptID, Salary
FROM EMPLOYEE E
WHERE Salary >
(
```

SaleID **Product** **Amount**

```
   SELECT AVG(Salary)
   FROM EMPLOYEE
   WHERE DeptID = E.DeptID
);
```

Sample Output

EmpID	EmpName	ProjectID
101	Rahul	P1
102	Priya	P2
103	Arjun	NULL
104	Sneha	P3

5. APPLY ALL TYPES OF JOIN OPERATIONS

EMPLOYEE

EmpID	EmpName	ProjectID
101	Rahul	P1
102	Priya	P2
103	Arjun	NULL
104	Sneha	P3

PROJECT

ProjectID	ProjectName
P1	Banking
P2	Hospital
P4	E-Commerce

(a) INNER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
INNER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Output

EmpName	ProjectName
Rahul	Banking
Priya	Hospital

(b) LEFT OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
```

```
FROM EMPLOYEE E
LEFT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Output

EmpName	ProjectName
Rahul	Banking
Priya	Hospital
Arjun	NULL
Sneha	NULL

(c) RIGHT OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
RIGHT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Output

EmpName	ProjectName
Rahul	Banking
Priya	Hospital
NULL	E-Commerce

(d) FULL OUTER JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Output

EmpName	ProjectName
Rahul	Banking
Priya	Hospital
Arjun	NULL
Sneha	NULL
NULL	E-Commerce

(e) CROSS JOIN

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
CROSS JOIN PROJECT P;
```

Output (Sample)

EmpName	ProjectName
Rahul	Banking
Rahul	Hospital
Rahul	E-Commerce
Priya	Banking
Priya	Hospital
Priya	E-Commerce
Arjun	Banking
Arjun	Hospital

EmpName	ProjectName
Arjun	E-Commerce
Sneha	Banking
Sneha	Hospital
Sneha	E-Commerce

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

Tables

CUSTOMER

CustomerID	CustomerName	AccountID
101	Rahul	A101
102	Priya	A102
103	Arjun	A103

ACCOUNT

AccountID	BranchID	Balance
A101	B1	50000
A102	B2	75000
A103	B1	40000

TRANSACTION

TransactionID	AccountID	Amount	TransactionType
T001	A101	5000	Deposit
T002	A101	2000	Withdrawal
T003	A102	10000	Deposit

BRANCH

BranchID	BranchName	City
B1	Anna Nagar Branch	Chennai
B2	T. Nagar Branch	Chennai

SQL Query

```
SELECT
   C.CustomerID,
   C.CustomerName,
   A.AccountID,
   A.Balance,
   T.TransactionID,
   T.Amount,
   T.TransactionType,
   B.BranchName,
   B.City
FROM CUSTOMER C
INNER JOIN ACCOUNT A
   ON C.AccountID = A.AccountID
```

EmpName **ProjectName**

```
INNER JOIN TRANSACTION T
   ON A.AccountID = T.AccountID
INNER JOIN BRANCH B
   ON A.BranchID = B.BranchID;
```

Output

CustomerID	CustomerName	AccountID	Balance	TransactionID	Amount	TransactionType	BranchName	City
101	Rahul	A101	50000	T001	5000	Deposit	Anna Nagar Branch	Chennai
101	Rahul	A101	50000	T002	2000	Withdrawal	Anna Nagar Branch	Chennai
102	Priya	A102	75000	T003	10000	Deposit	T. Nagar Branch	Chennai

7)A retail store needs a daily sales summary. Write an SQL query using GROUP BY and HAVING to report the total sales per category exceeding Rs. 10,000.

SALES Table

SaleID	Category	Amount
1	Electronics	6000
2	Electronics	7000
3	Clothing	5000
4	Clothing	3000
5	Grocery	12000
6	Grocery	4000

SQL Query

```
SELECT Category, SUM(Amount) AS TotalSales
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

Output

Category	TotalSales (Rs.)
Electronics	13000
Grocery	16000

8)For a library system scenario, construct Tuple Relational Calculus (TRC) expressions to find members who borrowed books from all genres.

Relations

MEMBER

MemberID	MemberName
M1	Rahul
M2	Priya
M3	Arjun

BOOK

BookID	Title	Genre
B1	DBMS	Technology
B2	Hamlet	Literature
B3	Physics	Science

BORROW

MemberID	BookID
M1	B1
M1	B2
M1	B3
M2	B1
M2	B2
M3	B1

Tuple Relational Calculus (TRC) Expression

{ M.MemberName |
  MEMBER(M) ∧
  ∀G (∃B (BOOK(B) ∧ B.Genre = G) →
      ∃BR (BORROW(BR) ∧
          BR.MemberID = M.MemberID ∧
          ∃BK (BOOK(BK) ∧
              BK.BookID = BR.BookID ∧
              BK.Genre = G)))
}

9)Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

Create the CATEGORY Table

```
CREATE TABLE CATEGORY (
   CategoryID INT PRIMARY KEY,
   CategoryName VARCHAR(50) NOT NULL
);
```

Create the PRODUCT Table with CHECK and FOREIGN KEY Constraints

```
CREATE TABLE PRODUCT (
   ProductID INT PRIMARY KEY,
   ProductName VARCHAR(50) NOT NULL,
   Price DECIMAL(10,2) CHECK (Price > 0),
   Stock INT CHECK (Stock >= 0),
   CategoryID INT,
   FOREIGN KEY (CategoryID)
   REFERENCES CATEGORY(CategoryID)
);
```

Insert Valid Records

```
INSERT INTO CATEGORY VALUES
(101, 'Electronics'),
(102, 'Furniture');

INSERT INTO PRODUCT VALUES
(1, 'Laptop', 55000, 10, 101),
(2, 'Chair', 3000, 25, 102);
```

Output

2 rows inserted into CATEGORY.

2 rows inserted into PRODUCT.

Demonstrate CHECK Constraint

```
INSERT INTO PRODUCT
VALUES (3, 'Mobile', -15000, 5, 101);
```

Output

ERROR:
CHECK constraint violated.
Price must be greater than 0.

Demonstrate FOREIGN KEY Constraint

```
INSERT INTO PRODUCT
VALUES (4, 'Table', 5000, 15, 105);
```

Output

ERROR:
Cannot add or update a child row.
FOREIGN KEY constraint fails.
(CategoryID 105 does not exist in CATEGORY table.)

Final PRODUCT Table

ProductID	ProductName	Price	Stock	CategoryID
1	Laptop	55000	10	101
2	Chair	3000	25	102

10)Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

Relational Schema

EMPLOYEE

EmployeeID	EmployeeName	Dept	Designation
101	Rahul	HR	Manager
102	Priya	IT	Developer

ATTENDANCE

AttendanceID	EmployeeID	Month	DaysPresent
A1	101	January	26
A2	102	January	24

PERFORMANCE

PerformanceID	EmployeeID	Month	Rating
P1	101	January	4.8
P2	102	January	4.5

SQL DDL Statements

```
CREATE TABLE EMPLOYEE (
   EmployeeID INT PRIMARY KEY,
   EmployeeName VARCHAR(50),
   Dept VARCHAR(30),
   Designation VARCHAR(30)
);

CREATE TABLE ATTENDANCE (
   AttendanceID VARCHAR(10) PRIMARY KEY,
   EmployeeID INT,
   Month VARCHAR(20),
   DaysPresent INT,
   FOREIGN KEY (EmployeeID)
   REFERENCES EMPLOYEE(EmployeeID)
);
```

```
CREATE TABLE PERFORMANCE (
   PerformanceID VARCHAR(10) PRIMARY KEY,
   EmployeeID INT,
   Month VARCHAR(20),
   Rating DECIMAL(3,1),
   FOREIGN KEY (EmployeeID)
   REFERENCES EMPLOYEE(EmployeeID)
);
```

Materialized View for Monthly Attendance Summary

```
CREATE MATERIALIZED VIEW Monthly_Attendance AS
SELECT EmployeeID,
      Month,
      SUM(DaysPresent) AS TotalDaysPresent
FROM ATTENDANCE
GROUP BY EmployeeID, Month;
```

Materialized View for Monthly Performance Summary

```
CREATE MATERIALIZED VIEW Monthly_Performance AS
SELECT EmployeeID,
      Month,
      AVG(Rating) AS AverageRating
FROM PERFORMANCE
GROUP BY EmployeeID, Month;
```

Sample Output

Monthly_Attendance

EmployeeID	Month	TotalDaysPresent
101	January	26
102	January	24

Monthly_Performance

EmployeeID	Month	AverageRating
101	January	4.8
102	January	4.5